

University of Tehran
College of Engineering
School of Electrical and
Computer Engineering

Machine Learning

Assignment 2

Name and Surname
Rouja Aghajani

Student Number
810101380

November 21, 2025

Contents

1 Abstract **1**

2 Question 1: Line Search Optimization Problem **3**

2.1 (a) General Form of the Line-Search Optimization Problem . . 3

2.2 (b) Descent Direction and Necessary Condition 3

2.3 (c) Common Methods for Selecting the Step Length 4

2.4 (d) Quadratic Function Example 5

2.4.1 1. Showing that $p^{(t)}$ is a Descent Direction 5

2.4.2 2. Forming the One-Dimensional Function $\varphi(\alpha)$ . . . 6

2.4.3 3. Computing the Exact Line-Search Minimizer α^* . . 7

3 Question 2: Support Vector Machine (SVM) — Primal, Lagrangian, Dual (and extensions) **8**

3.1 (a) Hard-margin SVM primal problem 8

3.2 (b) Lagrangian and stationarity conditions 8

3.3 (c) Dual problem (in α only) 9

3.4 (d) KKT conditions, complementary slackness, and support vectors 10

3.5 (e) Soft-margin SVM (slack variables) and its dual 11

3.6 (f) Kernelization and kernelized decision function 12

4 Question 3: Bilevel Optimization (scalar example) **14**

4.1 Problem statement 14

5 Question 4: Multi-Objective Optimization — Pareto Front **20**

5.1 (a) Pareto optimality 20

5.2 (b) Weighted-sum scalarization 20

5.2.1 (b.1) Minimizer $(x(w), y(w))$ 20
5.2.2 (b.2) Pareto set justification 21
5.3 (c) Parametric Pareto front and relation between $u = f_1$ and $v = f_2$ 21
5.4 (d) Epsilon-constraint and KKT 22
5.5 (e) Numerical values for selected weights 22

6 Question 5: Genetic Algorithm for 2x2 Jigsaw Puzzle 24
6.1 Problem Setup 24
6.2 Fitness Function 24
6.3 Genetic Algorithm Steps 25
6.4 Results 25
6.4.1 Generations vs Best Fitness 25

7 Question 6: Simulated Annealing for the Travelling Salesman Problem (TSP) 27
7.1 Problem Formulation 28
7.2 Simulated Annealing (SA) 28
7.3 Neighborhood Operators 29
7.4 Initial Solutions 29
7.5 Results 29
7.6 Best Tour and Length (Swap, Random Initialization) 30
7.7 Best Tour and Length (2-opt, Greedy Initialization) 30
7.8 Discussion 30
7.9 Cooling Schedules and Parameters 30
7.10 Convergence Plot 31
7.11 Visualization of the Best Tour 31
7.12 Conclusion 32

8 Question 7: Batch vs. Stochastic Gradient Descent 33
8.1 (a) Computational load per iteration 33

8.2 (b) Noise (variance) in the gradient estimation 34

9 Question 8: Steepest Descent Direction via First-Order Taylor 36

10 Question 9: The pseudoinverse solution is the least-squares minimizer 38

11 Question 10: Newton and Quasi-Newton Methods 41

11.1 (a) How is the Newton step computed? 41

11.2 (b) When can Newton's method fail to converge to a minimum? 41

11.3 (c) When is the Newton direction not a descent direction? . . . 42

11.4 (d) What is the main idea of Quasi-Newton methods? 42

11.5 (e) How do Quasi-Newton methods approximate the Hessian without explicit second derivatives? 43

12 Question 11: Quadratic Convergence of Newton's Method 45

12.1 (a) Definition and comparison with linear convergence 45

12.2 (b) Newton direction approximates the exact displacement (and quadratic convergence sketch) 45

13 Question 12: Rosenbrock Function — Gradient Descent vs Newton 50

13.1 (a) Gradient and Hessian 50

13.2 (b) Optimization Algorithms 51

13.3 (c) Convergence Considerations 51

13.4 (d) Numerical Results 52

13.5 (e) Discussion 53

13.6 (f) Conclusion 53

14 Question 13: Equality-Constrained Optimization via Lagrange Multipliers & Newton–Raphson 55

14.1 (a) Lagrangian Formulation 55

14.2 (b) Stationarity Conditions 55

14.3 (c) Nonlinear System for Newton–Raphson 56
14.4 (d) Newton–Raphson Update and Jacobian 56
14.5 (e) Newton–Raphson Iterations from $w_0 = [0, 0, 0]^\top$ 57
14.6 Remarks 59

15 Question 14: Step-Size Strategies in Gradient Descent 60
15.1 (a) Step-Size Selection Methods 60
15.2 (b) Implementation for the Rosenbrock Function 61
15.3 (c) Comparison of Results 62

16 Refrences 63

List of Figures

1 Fitness vs Generation 27
2 Convergence of Simulated Annealing with Swap and 2-opt Operators 31
3 Convergence of Simulated Annealing with Swap and 2-opt Operators 53

List of Tables

1 Best fitness per generation. 26

2 Newton–Raphson iterations for the constrained system 58

Abstract

This assignment provides an in-depth exploration of optimization techniques and machine learning algorithms. The core focus is on methods like line search, gradient descent, and Newton's method, which are fundamental to solving optimization problems in machine learning. We begin by analyzing the line search optimization problem, understanding descent directions, and exploring different step-size strategies like backtracking and exact line search. These concepts are applied to Support Vector Machines (SVM), emphasizing the relationship between primal and dual formulations.

The assignment also delves into advanced algorithms, including Genetic Algorithms (GA) for solving combinatorial problems like jigsaw puzzles, and Simulated Annealing (SA) applied to the Traveling Salesman Problem (TSP). We examine how these algorithms utilize evolutionary processes and probabilistic decision-making to converge to optimal solutions. The impact of cooling schedules, selection mechanisms, and neighboring operators in these algorithms is highlighted.

In the context of gradient-based optimization, we compare Batch Gradient Descent (BGD) and Stochastic Gradient Descent (SGD), focusing on their computational efficiency, noise characteristics, and convergence behavior. The assignment also investigates step-size strategies like adaptive methods (AdaGrad, RMSProp) and their effects on optimization efficiency. Finally, we analyze the convergence properties of Newton's method and its quasi-variant, BFGS, particularly in terms of quadratic convergence and computational cost.

Overall, this assignment provides a comprehensive introduction to a wide range of optimization techniques, preparing us to tackle both theoretical and

practical challenges in machine learning and optimization.

Question 1: Line Search Optimization Problem

2.1 (a) General Form of the Line-Search Optimization Problem

In line-search optimization methods, at each iteration t we have a current point $x^{(t)}$ and we pick a search direction $p^{(t)}$. The algorithm then looks for a suitable step length $\alpha^{(t)} > 0$ so that the next point is obtained by

$$x^{(t+1)} = x^{(t)} + \alpha^{(t)} p^{(t)}.$$

This means we restrict the optimization to the one-dimensional function

$$\varphi(\alpha) := f\left(x^{(t)} + \alpha p^{(t)}\right),$$

and then line search attempts to solve

$$\alpha^{(t)} = \arg\min_{\alpha \geq 0} \varphi(\alpha).$$

Role of the search direction: The vector $p^{(t)}$ determines *which direction* we move from $x^{(t)}$, ideally toward a region of lower function value.

Role of the step length: The scalar $\alpha^{(t)}$ determines *how far* we move along the chosen direction. A good step length should decrease the function while keeping progress efficient.

2.2 (b) Descent Direction and Necessary Condition

A vector p is called a *descent direction* for a differentiable function f at a point x if moving a small step along p decreases the function value. Formally, p is a

descent direction if there exists $\epsilon > 0$ such that

$$f(x + \alpha p) < f(x), \qquad \forall \alpha \in (0, \epsilon).$$

A commonly used necessary (and sufficient for infinitesimal steps) condition is based on the directional derivative:

$$\nabla f(x)^\top p < 0.$$

If this inequality holds, then p is guaranteed to be a descent direction at x.

2.3 (c) Common Methods for Selecting the Step Length

Here are several well-known methods for determining the step length α:

- **Exact Line Search:** Minimize $\varphi(\alpha) = f(x + \alpha p)$ exactly or to high precision. This works well for simple functions (e.g., quadratics), but may be expensive in general.
- **Armijo (Backtracking) Line Search:** Start from an initial α_0 and repeatedly shrink α until the sufficient decrease condition

  $$f(x + \alpha p) \leq f(x) + c\alpha \nabla f(x)^\top p,$$

  with $0 < c < 1$, is satisfied.
- **Wolfe or Strong Wolfe Conditions:** Choose α that satisfies both the Armijo condition and a curvature condition, such as

  $$\nabla f(x + \alpha p)^\top p \geq c_2 \nabla f(x)^\top p,$$

  or (for strong Wolfe)

  $$\left|\nabla f(x + \alpha p)^\top p\right| \leq c_2 \left|\nabla f(x)^\top p\right|.$$

- **Fixed Step Length:** Use a constant step size determined beforehand. Simple but can be inefficient or unstable.
- **Barzilai–Borwein (BB) Method:** Approximates second-order information using previous steps. For example,

$$\alpha_{\text{BB1}} = \frac{s^\top s}{s^\top y}, \qquad \alpha_{\text{BB2}} = \frac{s^\top y}{y^\top y},$$

where $s = x^{(t)} - x^{(t-1)}$ and $y = \nabla f(x^{(t)}) - \nabla f(x^{(t-1)})$.

2.4 (d) Quadratic Function Example

We are given the function

$$f(x_1, x_2) = 2x_1^2 + x_1x_2 + 4x_2^2,$$

and at iteration t the current point and search direction are

$$x^{(t)} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \qquad p^{(t)} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}.$$

2.4.1 1. Showing that $p^{(t)}$ is a Descent Direction

First we compute the gradient of f:

$$\frac{\partial f}{\partial x_1} = 4x_1 + x_2, \qquad \frac{\partial f}{\partial x_2} = x_1 + 8x_2.$$

Evaluating at $x^{(t)} = (0, 1)^\top$:

$$\nabla f(x^{(t)}) = \begin{pmatrix} 4(0) + 1 \\ 0 + 8(1) \end{pmatrix} = \begin{pmatrix} 1 \\ 8 \end{pmatrix}.$$

Now we check the descent condition:

$$\nabla f(x^{(t)})^\top p^{(t)} = \begin{pmatrix} 1 & 8 \end{pmatrix} \begin{pmatrix} 1 \\ -2 \end{pmatrix} = 1(1) + 8(-2) = 1 - 16 = -15.$$

Since $-15 < 0$, the vector $p^{(t)}$ is indeed a descent direction at $x^{(t)}$.

2.4.2 2. Forming the One-Dimensional Function $\varphi(\alpha)$

We compute

$$x^{(t)} + \alpha p^{(t)} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} + \alpha \begin{pmatrix} 1 \\ -2 \end{pmatrix} = \begin{pmatrix} \alpha \\ 1 - 2\alpha \end{pmatrix}.$$

Thus

$$x_1(\alpha) = \alpha, \qquad x_2(\alpha) = 1 - 2\alpha.$$

Now substitute into f:

$$\begin{aligned} \varphi(\alpha) &= f\big(\alpha,\, 1 - 2\alpha\big) \\ &= 2(\alpha)^2 + \alpha(1 - 2\alpha) + 4(1 - 2\alpha)^2. \end{aligned}$$

We expand each term:

$$2\alpha^2 \quad \text{(first term)},$$

$$\alpha(1 - 2\alpha) = \alpha - 2\alpha^2,$$

$$4(1 - 2\alpha)^2 = 4(1 - 4\alpha + 4\alpha^2) = 4 - 16\alpha + 16\alpha^2.$$

Now sum them:

$$\begin{aligned} \varphi(\alpha) &= \big(2\alpha^2 - 2\alpha^2\big) + \alpha + 4 - 16\alpha + 16\alpha^2 \\ &= 16\alpha^2 - 15\alpha + 4. \end{aligned}$$

So the one-dimensional function has the quadratic form

$$\varphi(\alpha) = 16\alpha^2 - 15\alpha + 4.$$

2.4.3 3. Computing the Exact Line-Search Minimizer α^*

Since $\varphi(\alpha)$ is a quadratic polynomial of the form

$$\varphi(\alpha) = a\alpha^2 + b\alpha + c,$$

its minimum (for $a > 0$) occurs at

$$\alpha^* = -\frac{b}{2a}.$$

Here we have $a = 16$ and $b = -15$, so

$$\alpha^* = -\frac{-15}{2 \cdot 16} = \frac{15}{32}.$$

Numerically,

$$\alpha^* = 0.46875 > 0.$$

Therefore, the exact line-search step length is positive, as required.

Final Answer:

$$\boxed{\alpha^* = \frac{15}{32} > 0.}$$

Question 2: Support Vector Machine (SVM) — Primal, Lagrangian, Dual (and extensions)

We consider a binary classification dataset $\{(x_i, y_i)\}_{i=1}^n$ with $x_i \in \mathbb{R}^d$ and labels $y_i \in \{+1, -1\}$.

3.1 (a) Hard-margin SVM primal problem

The hard-margin SVM seeks a separating hyperplane with maximum margin. The primal optimization problem is

$$\begin{aligned} \min_{w\in\mathbb{R}^d,\, b\in\mathbb{R}} \quad & \frac{1}{2}\|w\|^2 \\ \text{subject to} \quad & y_i\left(w^\top x_i + b\right) \geq 1, \qquad i = 1, \ldots, n. \end{aligned} \tag{P}$$

The objective $\frac{1}{2}\|w\|^2$ corresponds to maximizing the margin $2/\|w\|$. The constraints enforce that every sample is correctly classified with margin at least 1.

3.2 (b) Lagrangian and stationarity conditions

Introduce Lagrange multipliers $\alpha_i \geq 0$ (one per inequality constraint). The (primal) Lagrangian is

$$\mathcal{L}(w, b, \alpha) \;=\; \frac{1}{2}\|w\|^2 \;-\; \sum_{i=1}^n \alpha_i\big(y_i(w^\top x_i + b) - 1\big).$$

Stationarity (first-order) conditions are obtained by setting derivatives of $\mathcal{L}$ w.r.t. primal variables w and b to zero.

Derivative w.r.t. w:

$$\nabla_w \mathcal{L} = w - \sum_{i=1}^{n} \alpha_i y_i x_i \;=\; 0 \quad \Longrightarrow \quad \boxed{w \;=\; \sum_{i=1}^{n} \alpha_i y_i x_i \,.}$$

Derivative w.r.t. b:

$$\frac{\partial \mathcal{L}}{\partial b} = -\sum_{i=1}^{n} \alpha_i y_i \;=\; 0 \quad \Longrightarrow \quad \boxed{\sum_{i=1}^{n} \alpha_i y_i \;=\; 0\,.}$$

These two stationarity conditions eliminate w and impose a linear equality constraint on the α_i.

3.3 (c) Dual problem (in α only)

To obtain the dual, substitute the stationarity expression $w = \sum_i \alpha_i y_i x_i$ into the Lagrangian and simplify. First compute:

$$\frac{1}{2}\|w\|^2 = \frac{1}{2}\left(\sum_{i=1}^{n} \alpha_i y_i x_i\right)^{\top}\left(\sum_{j=1}^{n} \alpha_j y_j x_j\right) = \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^{\top} x_j.$$

Also the term $-\sum_i \alpha_i(-1)$ contributes $+\sum_i \alpha_i$. Substituting into $\mathcal{L}$ and using $\sum_i \alpha_i y_i \;=\; 0$ to remove b-term yields the dual objective (we write it as a maximization in α):

$$\begin{aligned} \max_{\alpha \in \mathbb{R}^n} \quad & \sum_{i=1}^{n} \alpha_i \;-\; \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^{\top} x_j \\ \text{subject to} \quad & \alpha_i \geq 0 \quad (i = 1, \ldots, n), \qquad \sum_{i=1}^{n} \alpha_i y_i = 0. \end{aligned} \tag{D}$$

This is the standard SVM dual: a convex quadratic program in α. Often written equivalently as the minimization of the negative of the objective.

3.4 (d) KKT conditions, complementary slackness, and support vectors

Since the primal and dual are convex and satisfy Slater's condition (for separable data), the Karush–Kuhn–Tucker (KKT) conditions are necessary and sufficient for optimality. The KKT conditions consist of:

1. **Primal feasibility:** $y_i(w^\top x_i + b) \geq 1$ for all i.
2. **Dual feasibility:** $\alpha_i \geq 0$ for all i.
3. **Stationarity:** $w = \sum_{i=1}^n \alpha_i y_i x_i$ and $\sum_{i=1}^n \alpha_i y_i = 0$.
4. **Complementary slackness:** $\alpha_i \big(y_i(w^\top x_i + b) - 1\big) = 0$ for all i.

Interpretation and support vectors: - Complementary slackness implies that for any index i either $\alpha_i = 0$ or the corresponding constraint is active $y_i(w^\top x_i + b) = 1$. - Points with $\alpha_i > 0$ must satisfy $y_i(w^\top x_i + b) = 1$; these points lie exactly on the margin hyperplanes and are called *support vectors*. They are the only training points that determine w. - Points with $\alpha_i = 0$ are *non-support* points and do not directly affect w.

Recovering w and b from $\alpha^\star$: - Once the dual is solved and we have $\alpha^\star$, recover

$$w^\star = \sum_{i=1}^n \alpha_i^\star y_i x_i.$$

- To obtain $b^\star$ pick any support vector x_k with $\alpha_k^\star > 0$. Then the margin equality holds:

$$y_k\big((w^\star)^\top x_k + b^\star\big) = 1 \quad \Longrightarrow \quad b^\star = y_k - (w^\star)^\top x_k.$$

In practice one often averages b over all support vectors with $0 < \alpha_i^\star$ to reduce numerical error:

$$b^\star = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \Big(y_i - (w^\star)^\top x_i \Big), \quad \mathcal{S} = \{ i : \alpha_i^\star > 0 \}.$$

3.5 (e) Soft-margin SVM (slack variables) and its dual

When the data are not linearly separable, the soft-margin SVM introduces slack variables $\xi_i \geq 0$ and a regularization parameter $C > 0$ that trades off margin size vs. slack (misclassification).

Soft-margin primal (hinge-loss form):

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|^2 + C \sum_{i=1}^{n} \xi_i \\ \text{subject to} \quad & y_i(w^\top x_i + b) \geq 1 - \xi_i, \qquad \xi_i \geq 0, \ i = 1, \ldots, n. \end{aligned} \tag{P}$$

soft

Introduce dual multipliers $\alpha_i \geq 0$ for the first constraints and $\mu_i \geq 0$ for the nonnegativity $\xi_i \geq 0$. The Lagrangian is

$$\mathcal{L}(w, b, \xi, \alpha, \mu) = \frac{1}{2}\|w\|^2 + C \sum_i \xi_i - \sum_i \alpha_i \big(y_i(w^\top x_i + b) - 1 + \xi_i \big) - \sum_i \mu_i \xi_i.$$

Stationarity conditions:

$$\nabla_w \mathcal{L} = 0 \implies w = \sum_{i=1}^{n} \alpha_i y_i x_i,$$

$$\frac{\partial \mathcal{L}}{\partial b} = 0 \implies \sum_{i=1}^{n} \alpha_i y_i = 0,$$

$$\frac{\partial \mathcal{L}}{\partial \xi_i} = 0 \implies C - \alpha_i - \mu_i = 0 \quad \implies \quad \alpha_i \leq C \quad (\text{since } \mu_i \geq 0).$$

Eliminating $w,\ b,\ \xi$ and using the above yields the dual:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^{n} \alpha_i \; - \; \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^\top x_j \\ \text{subject to} \quad & 0 \le \alpha_i \le C, \quad i = 1, \ldots, n, \qquad \sum_{i=1}^{n} \alpha_i y_i = 0. \end{aligned} \tag{D}$$

soft

Thus for soft-margin SVM the only change from the hard-margin dual is the upper bound $\alpha_i \le C$. The bound arises from stationarity on ξ_i and the multiplier μ_i.

Interpretation of bounds: - If $\alpha_i = 0$ the point is non-support (not active). - If $0 < \alpha_i < C$ the point lies exactly on the margin (correctly classified and on margin): $y_i(w^\top x_i + b) = 1$. - If $\alpha_i = C$ the point is either inside the margin or misclassified (slack $\xi_i > 0$).

3.6 (f) Kernelization and kernelized decision function

The dual problems (both hard and soft) depend only on the training data through dot products $x_i^\top x_j$. This enables the *kernel trick*: replace $x_i^\top x_j$ by a kernel function $k(x_i, x_j)$ which implicitly corresponds to an inner product in some feature space $\phi(\cdot)$, i.e. $k(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$.

Kernelized dual (soft-margin shown, similarly for hard):

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^{n} \alpha_i \; - \; \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j k(x_i, x_j) \\ \text{subject to} \quad & 0 \le \alpha_i \le C, \quad i = 1, \ldots, n, \qquad \sum_{i=1}^{n} \alpha_i y_i = 0. \end{aligned}$$

Kernelized decision function: Once $\alpha^\star$ and $b^\star$ are obtained, the classifier for a test point x is

$$f(x) \;=\; \text{sign}\Big(\sum_{i=1}^{n} \alpha_i^\star y_i\, k(x_i, x) \;+\; b^\star\Big).$$

Only support vectors with $\alpha_i^\star > 0$ contribute to the sum, so this is typically written summing over $\mathcal{S} = \{i : \alpha_i^\star > 0\}$.

Computing $b^\star$ in kernel form: For any support vector x_k with $0 < \alpha_k^\star < C$,

$$y_k \;=\; \sum_{i\in\mathcal{S}} \alpha_i^\star y_i k(x_i, x_k) + b^\star \quad\Longrightarrow\quad b^\star = y_k - \sum_{i\in\mathcal{S}} \alpha_i^\star y_i k(x_i, x_k).$$

Again, it is common to average this over all such support vectors.

Final summary of key formulas:

- Hard-margin primal: $\min_{w,b} \frac{1}{2}\|w\|^2$ s.t. $y_i(w^\top x_i + b) \geq 1$.
- Dual (hard): $\max_\alpha \sum_i \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i\alpha_j y_i y_j x_i^\top x_j$ s.t. $\alpha_i \geq 0,\ \sum_i \alpha_i y_i = 0$.
- Soft-margin primal: $\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_i \xi_i$ s.t. $y_i(w^\top x_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0$.
- Dual (soft): same objective as hard but with $0 \leq \alpha_i \leq C$ and $\sum_i \alpha_i y_i = 0$.
- Recover $w^\star = \sum_i \alpha_i^\star y_i x_i$ (or in kernel form use kernel in decision function).
- Decision function (kernelized): $\text{sign}\big(\sum_i \alpha_i^\star y_i k(x_i, x) + b^\star\big)$.

Question 3: Bilevel Optimization (scalar example)

4.1 Problem statement

The bilevel problem is:

$$\begin{aligned} \min_{x\in\mathbb{R}} \quad & F(x,y) = (x-2)^2 + (y-3)^2 \\ \text{s.t.} \quad & x \geq 0, \\ & y \in \arg\min_{y\in\mathbb{R}} \; f(x,y) := (y-x^2)^2 + y \quad \text{s.t. } y \geq 0. \end{aligned}$$

We solve this problem by first solving the lower-level (follower) problem for fixed x, then substituting the follower's optimal response $y^*(x)$ into the leader objective and minimizing over $x \geq 0$.

(a) Bilevel formulation (upper and lower)

Upper (leader):

$$\min_{x\in\mathbb{R}} \; F(x,y) = (x-2)^2 + (y-3)^2 \quad \text{s.t. } x \geq 0, \; y \text{ solves the lower problem.}$$

Lower (follower), for given x:

$$\min_{y\in\mathbb{R}} \; f(x,y) = (y-x^2)^2 + y \quad \text{s.t. } y \geq 0.$$

(b) Solve the follower for fixed x: obtain $y^*(x)$.

Consider the follower objective (for fixed x):

$$f(x,y) = (y-x^2)^2 + y.$$

This is a scalar function in y with derivative

$$\frac{\partial f}{\partial y} = 2(y - x^2) + 1 = 2y - 2x^2 + 1.$$

Set derivative to zero (unconstrained minimizer):

$$2y - 2x^2 + 1 = 0 \quad \Longrightarrow \quad y = x^2 - \tfrac{1}{2}.$$

Check second derivative:

$$\frac{\partial^2 f}{\partial y^2} = 2 > 0,$$

so $f(x, \cdot)$ is strictly convex in y and the stationary point is the unique unconstrained minimizer.

Now account for the constraint $y \geq 0$. The constrained minimizer is the projection of the unconstrained minimizer onto $[0, \infty)$:

$$y^*(x) \;=\; \begin{cases} 0, & \text{if } x^2 - \frac{1}{2} < 0 \quad (\,|x| < \frac{1}{\sqrt{2}}\,), \\ x^2 - \frac{1}{2}, & \text{if } x^2 - \frac{1}{2} \geq 0 \quad (\,|x| \geq \frac{1}{\sqrt{2}}\,). \end{cases}$$

Since the leader has the constraint $x \geq 0$, we only need the branch for $x \geq 0$:

$$y^*(x) = \begin{cases} 0, & 0 \leq x < \frac{1}{\sqrt{2}}, \\ x^2 - \frac{1}{2}, & x \geq \frac{1}{\sqrt{2}}. \end{cases}$$

(c) Substitute $y^*(x)$ into the leader objective and reduce to a single-variable problem.

Define the reduced function $\Phi(x) := F(x, y^*(x))$. Using the two branches of $y^*(x)$ we obtain a piecewise function.

Branch 1: $0 \le x < \frac{1}{\sqrt{2}}$. Here $y^*(x) = 0$, so

$$\Phi(x) = (x-2)^2 + (0-3)^2 = (x-2)^2 + 9.$$

Branch 2: $x \ge \frac{1}{\sqrt{2}}$. Here $y^*(x) = x^2 - \frac{1}{2}$, so

$$\begin{aligned}\Phi(x) &= (x-2)^2 + \left(x^2 - \tfrac{1}{2} - 3\right)^2 \\ &= (x-2)^2 + (x^2 - 3.5)^2.\end{aligned}$$

Thus the bilevel problem reduces to

$$\min_{x \ge 0} \Phi(x) = \min\left\{ \min_{0 \le x < \frac{1}{\sqrt{2}}} \left[(x-2)^2 + 9\right],\ \min_{x \ge \frac{1}{\sqrt{2}}} \left[(x-2)^2 + (x^2 - 3.5)^2\right] \right\}.$$

(d) Find the optimizer (x^*, y^*)**.**

We treat the two branches separately.

Branch 1: $0 \le x < \frac{1}{\sqrt{2}}$

$$\Phi(x) = (x-2)^2 + 9.$$

This is a simple convex quadratic in x with derivative $\Phi'(x) = 2(x-2)$ and minimizer (unconstrained) at $x = 2$. Because $x = 2$ is not in the interval $[0, \frac{1}{\sqrt{2}})$, the minimum on this interval is attained at the right endpoint $x = \frac{1}{\sqrt{2}}$. Evaluate:

$$\Phi\left(\tfrac{1}{\sqrt{2}}\right) = \left(\tfrac{1}{\sqrt{2}} - 2\right)^2 + 9 = \left(2 - \tfrac{1}{\sqrt{2}}\right)^2 + 9 \approx 10.6715728753.$$

Branch 2: $x \ge \frac{1}{\sqrt{2}}$

$$\Phi(x) = (x-2)^2 + (x^2 - 3.5)^2.$$

Differentiate w.r.t. x:

$$\Phi'(x) = 2(x-2) + 2(x^2 - 3.5) \cdot 2x = 2(x-2) + 4x(x^2 - 3.5).$$

Set $\Phi'(x) = 0$. Dividing by 2 gives:

$$(x-2) + 2x(x^2 - 3.5) = 0 \quad \Longrightarrow \quad 2x^3 - 7x + x - 2 = 0$$

(rearrange carefully). Simplify to the standard cubic:

$$4(x^3 - 3x - 1) = 0 \quad \Longrightarrow \quad x^3 - 3x - 1 = 0.$$

So stationary points solve

$$x^3 - 3x - 1 = 0.$$

This cubic has three real roots. Using the trigonometric closed-form (set $x = 2\cos\theta$):

$$x^3 - 3x = 2\cos 3\theta,$$

so the equation $x^3 - 3x - 1 = 0$ becomes $2\cos 3\theta - 1 = 0$, i.e.

$$\cos 3\theta = \tfrac{1}{2} \quad \Longrightarrow \quad 3\theta = \pm\tfrac{\pi}{3} + 2k\pi.$$

One of the corresponding roots (the positive one) is

$$\boxed{x^* = 2\cos\left(\tfrac{\pi}{9}\right) \approx 1.8793852416.}$$

(This is the relevant root because we require $x \geq 0$ and $x \geq \frac{1}{\sqrt{2}}$.)

Compute the follower response for this x^*:

$$y^* = (x^*)^2 - \tfrac{1}{2} = \left(2\cos(\tfrac{\pi}{9})\right)^2 - \tfrac{1}{2} = 4\cos^2\left(\tfrac{\pi}{9}\right) - \tfrac{1}{2} \approx 3.0320888863.$$

Evaluate the leader objective at this point:

$$\Phi(x^*) \approx 0.0155776166,$$

which is much lower than the best value on Branch 1 (≈10.6716). Therefore the global minimizer of the bilevel problem (with $x \geq 0$) is

$$\boxed{x^* = 2\cos\left(\tfrac{\pi}{9}\right) \approx 1.8793852416, \qquad y^* = (x^*)^2 - \tfrac{1}{2} \approx 3.0320888863.}$$

(One can verify boundary candidates such as $x = 0$ yield larger objective values: $\Phi(0) = 13$.)

(e) Regularity discussion (convexity, uniqueness, validity of substitution)

- *Convexity of the follower problem:* For fixed x, the follower objective in y is

  $$f(x, y) = (y - x^2)^2 + y,$$

  whose second derivative w.r.t. y is $\partial^2 f/\partial y^2 = 2 > 0$. Hence for each fixed x the lower-level problem is strictly convex in y. Strict convexity guarantees uniqueness of the unconstrained minimizer.

- *Constraint on the follower:* The follower has the simple convex constraint $y \geq 0$. Because the unconstrained minimizer is unique and the feasible set is convex, the constrained minimizer $y^*(x)$ is unique (it is the projection of the unconstrained minimizer onto $[0, \infty)$).

- *Validity of substitution:* Since for every admissible x the follower problem has a unique solution $y^*(x)$, it is valid to substitute $y^*(x)$ into the leader's objective and reduce the bilevel program to a single-level optimization problem in x. This substitution is the standard approach when the lower-level optimal solution is single-valued (a mapping).

- *Continuity and differentiability of $y^*(x)$:* The mapping $y^*(x)$ is continuous everywhere but not differentiable at the threshold $x = \frac{1}{\sqrt{2}}$ where the unconstrained minimizer hits the constraint boundary (i.e. when $x^2 - \frac{1}{2} = 0$). Because of this, the reduced objective $\Phi(x)$ is continuous everywhere and differentiable except possibly at $x = \frac{1}{\sqrt{2}}$. In our problem the global minimizer lies in the smooth region $x > \frac{1}{\sqrt{2}}$, so differentiation-based optimality conditions were valid there.

- *Summary:* Strict convexity of the follower's objective and convex feasible set ensure uniqueness of the follower's response $y^*(x)$ for each x, which justifies reducing the bilevel problem to a single-level problem by substitution. The only technical subtlety is the possible nonsmoothness of the reduced objective at parameter values where the follower solution switches from the interior to the boundary; these points must be handled by checking both branches (as we did).

Final numerical answer (rounded):

$$x^* \approx 1.8793852416, \qquad y^* \approx 3.0320888863, \qquad \Phi(x^*) \approx 0.0155776166.$$

Question 4: Multi-Objective Optimization — Pareto Front

We consider the two-objective problem on $(x, y) \in \mathbb{R}^2$ with

$$f_1(x, y) = (x - 2)^2 + y^2, \qquad f_2(x, y) = x^2 + (y - 3)^2.$$

5.1 (a) Pareto optimality

A point (x^*, y^*) is *Pareto optimal* if there is no other feasible point (x, y) such that

$$f_i(x, y) \leq f_i(x^*, y^*) \text{ for } i = 1, 2,$$

with at least one strict inequality. In other words, improving one objective must worsen the other.

5.2 (b) Weighted-sum scalarization

For $w \in [0, 1]$, define the scalarized objective

$$F_w(x, y) = w f_1(x, y) + (1 - w) f_2(x, y).$$

5.2.1 (b.1) Minimizer $(x(w), y(w))$

Expand F_w:

$$\begin{aligned} F_w(x, y) &= w\big((x - 2)^2 + y^2\big) + (1 - w)\big(x^2 + (y - 3)^2\big) \\ &= x^2 - 4wx + 4w + y^2 - 6(1 - w)y + 9(1 - w). \end{aligned}$$

Set gradients to zero:

$$\frac{\partial F_w}{\partial x} = 2x - 4w = 0 \quad \Rightarrow \quad x(w) = 2w,$$

$$\frac{\partial F_w}{\partial y} = 2y - 6(1-w) = 0 \quad \Rightarrow \quad y(w) = 3(1-w).$$

Thus,

$$\boxed{x(w) = 2w, \qquad y(w) = 3(1-w), \quad w \in [0,1].}$$

5.2.2 (b.2) Pareto set justification

Since both f_1 and f_2 are convex, and weighted sums of convex functions remain convex, every minimizer of F_w is Pareto optimal. As w varies in $[0,1]$, these minimizers trace the entire Pareto set.

5.3 (c) Parametric Pareto front and relation between $u = f_1$ and $v = f_2$

Evaluate the objectives on $(x(w), y(w))$:

$$f_1(w) = (2w-2)^2 + (3(1-w))^2 = 13(1-w)^2,$$

$$f_2(w) = (2w)^2 + (3(1-w)-3)^2 = 13w^2.$$

Thus the Pareto front is

$$\boxed{P = \{(13(1-w)^2,\ 13w^2) : w \in [0,1]\}.}$$

Let $u = f_1$, $v = f_2$:

$$\sqrt{\frac{u}{13}} = 1 - w, \qquad \sqrt{\frac{v}{13}} = w.$$

Summing:

$$\boxed{\sqrt{u} + \sqrt{v} = \sqrt{13}.}$$

This is the implicit Pareto-front curve.

5.4 (d) Epsilon-constraint and KKT

Consider:

$$\min f_1(x, y) \quad \text{s.t.} \quad f_2(x, y) \leq t.$$

Lagrangian:

$$\mathcal{L}(x, y, \lambda) = f_1(x, y) + \lambda(f_2(x, y) - t), \qquad \lambda \geq 0.$$

KKT stationarity:

$$\nabla f_1(x, y) + \lambda \nabla f_2(x, y) = 0.$$

This is equivalent to minimizing a weighted sum:

$$f_1 + \lambda f_2 \quad \Longleftrightarrow \quad w f_1 + (1 - w) f_2,$$

with

$$w = \frac{1}{1 + \lambda}.$$

Thus every KKT point of the epsilon-constraint problem corresponds to a weighted-sum Pareto point.

5.5 (e) Numerical values for selected weights

Using

$$x(w) = 2w, \quad y(w) = 3(1 - w), \quad f_1 = 13(1 - w)^2, \quad f_2 = 13w^2,$$

we compute for $w \in \{0, 0.25, 0.5, 0.75, 1\}$:

w	$x(w)$	$y(w)$	$f_1(w)$	$f_2(w)$
0	0.0000	3.0000	13.0000	0.0000
0.25	0.5000	2.2500	7.3125	0.8125
0.5	1.0000	1.5000	3.2500	3.2500
0.75	1.5000	0.7500	0.8125	7.3125
1	2.0000	0.0000	0.0000	13.0000

Question 5: Genetic Algorithm for 2x2 Jigsaw Puzzle

6.1 Problem Setup

The 2x2 puzzle consists of 4 pieces, each represented by a 2×2 array:

$$\text{Piece}_i = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \quad i = 0, 1, 2, 3$$

The objective is to find the order (p_0, p_1, p_2, p_3) that minimizes the difference along adjacent edges.

6.2 Fitness Function

To measure compatibility, the difference along adjacent edges is computed as:

$$\text{fit}(piece_i, piece_j, side) = \sum |\text{edge of } piece_i - \text{edge of } piece_j|$$

The total difference for the puzzle is:

$$\text{total_diff} = \sum_{\text{all adjacent edges}} \text{fit}(piece_i, piece_j, side)$$

We define a positive fitness function:

$$\text{fitness} = \text{max_diff} - \text{total_diff}$$

A higher fitness indicates a better arrangement of pieces.

6.3 Genetic Algorithm Steps

The GA proceeds as follows:

1. **Initialization:** Generate a population of random arrangements.
2. **Fitness Evaluation:** Compute the fitness of each arrangement.
3. **Selection:** Choose better arrangements to produce the next generation (tournament selection).
4. **Crossover:** Combine two parent arrangements using single-point crossover.
5. **Mutation:** With a probability, swap two pieces in the arrangement.
6. **Replacement:** Replace the current population with the new generation and repeat the process.

6.4 Results

The GA was executed for 20 generations with a population of 10 arrangements. The best puzzle arrangement found is:

$$\text{Best puzzle arrangement (piece IDs)} = [0, 1, 2, 3]$$

6.4.1 Generations vs Best Fitness

The graph shows that the algorithm converges to the optimal arrangement after a few generations. The final arrangement corresponds to the lowest total edge differences and the highest fitness value.

Also as seen above, we reached highest fitness in multiple cases, but all had the same arrangement, meaning we found the arrangement that fits the most.

Generation	Best Fitness
0	83
1	83
2	87
3	89
4	89
5	89
6	89
7	89
8	89
9	89
10	89
11	89
12	89
13	89
14	89
15	87
16	87
17	89
18	89
19	87

Table 1: Best fitness per generation.

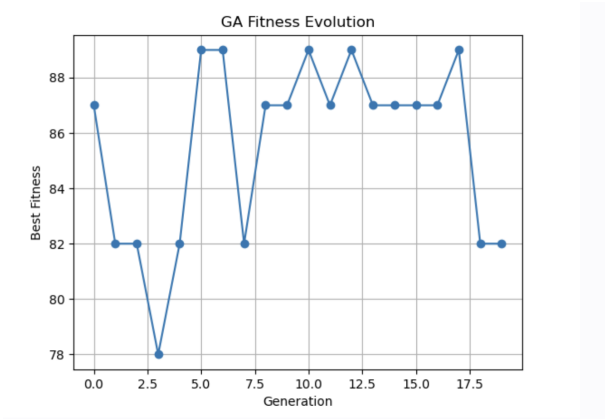


Figure 1: Fitness vs Generation

Question 6: Simulated Annealing for the Travelling Salesman Problem (TSP)

The Travelling Salesman Problem (TSP) is a well-known combinatorial optimization problem where the objective is to find the shortest possible route that visits a set of cities and returns to the origin city. Specifically, given N cities with Euclidean coordinates $c_1, c_2, \ldots, c_N \in \mathbb{R}^2$, the goal is to determine a permutation π of the cities such that the total tour length $L(\pi)$ is minimized. The tour length is defined as the sum of the Euclidean distances between consecutive cities, including the distance from the last city back to the first city.

The objective function is given by:

$$L(\pi) = \sum_{k=1}^{N} d(c_{\pi(k)}, c_{\pi(k+1)}), \quad c_{\pi(N+1)} \equiv c_{\pi(1)},$$

where $d(\cdot, \cdot)$ represents the Euclidean distance between two cities.

In this problem, we will apply Simulated Annealing (SA) to find an approximately optimal tour. We will implement the SA algorithm and explore various

design choices, including the neighborhood operator (swap vs. 2-opt), cooling schedules (geometric and linear), and initial solutions (random vs. greedy nearest-neighbor). Our goal is to evaluate how these factors influence the quality of the solution, the runtime, and the convergence behavior.

In the following sections, we will implement the algorithm and analyze the results, comparing the performance of different configurations and parameter settings. We will also examine the impact of the cooling schedule, neighborhood operator, and initial solution on the effectiveness of the algorithm.

7.1 Problem Formulation

Given N cities with Euclidean coordinates $c_1, c_2, \ldots, c_N \in \mathbb{R}^2$, the task is to find a permutation π that minimizes the total tour length $L(\pi)$:

$$L(\pi) = \sum_{k=1}^{N} d(c_{\pi(k)}, c_{\pi(k+1)}), \quad c_{\pi(N+1)} \equiv c_{\pi(1)}$$

where $d(\cdot, \cdot)$ represents the Euclidean distance between two cities.

The objective is to minimize the energy function $E(\pi) = L(\pi)$, where a shorter tour corresponds to lower energy.

7.2 Simulated Annealing (SA)

Simulated Annealing is a probabilistic optimization technique inspired by the annealing process in metallurgy. It involves exploring the solution space and gradually decreasing the ”temperature” to settle on a good solution. The algorithm proceeds as follows:

1. Initial Solution: Start with an initial solution (tour). 2. Neighbourhood Moves: Generate a neighboring solution by applying a move such as a swap or 2-opt. 3. Acceptance Probability: Use the Metropolis criterion to decide whether to accept the new solution.

$$P(\text{accept}) = \begin{cases} 1, & \Delta E \leq 0 \\ \exp(-\Delta E/T), & \Delta E > 0 \end{cases}$$

where $\Delta E = E_{\text{new}} - E_{\text{current}}$ is the change in energy, and T is the current temperature.

4. Cooling Schedule: Gradually reduce the temperature using one of the following schedules: - Geometric: $T_{k+1} = \alpha T_k$ - Linear: $T_{k+1} = T_k - \beta$ - Logarithmic: $T_k = \frac{T_0}{\log(1+k)}$

5. Stopping Criteria: The process stops when the temperature T drops below a minimum value T_{min} or after a certain number of iterations.

7.3 Neighborhood Operators

Two neighborhood operators were implemented:

- Swap: Two random cities are swapped in the tour.
- 2-opt: A segment of the tour is reversed to reduce the total tour length.

7.4 Initial Solutions

Two initial solutions were used:

- Random Initialization: A random permutation of the cities.
- Greedy Nearest Neighbor: Start from the first city and always visit the nearest unvisited city.

7.5 Results

We performed several experiments using the above setup. The results of two runs are summarized below:

7.6 Best Tour and Length (Swap, Random Initialization)

The best tour found using the swap operator with a random initialization is:

$$[4, 1, 6, 9, 7, 2, 8, 3, 5, 0]$$

The length of this tour is approximately 290.31 units.

7.7 Best Tour and Length (2-opt, Greedy Initialization)

The best tour found using the **2-opt operator** with a **greedy nearest neighbor initialization** is:

$$[0, 4, 1, 6, 9, 7, 2, 8, 3, 5]$$

The length of this tour is also approximately 290.31 units.

7.8 Discussion

Both the **swap** and **2-opt** neighborhood operators produced the same tour length, although the **2-opt operator** generally provides better local optimization. In this case, the performance of both operators was identical due to the structure of the randomly generated cities and the stopping criteria.

The greedy nearest neighbor initialization provided a slightly better starting point compared to a random initialization, but the final results were very similar. This is due to the nature of the problem, where the search space is vast and the cooling schedule is more influential in reaching a good solution.

7.9 Cooling Schedules and Parameters

The choice of the cooling schedule and parameters such as the initial temperature T_0, temperature decay rate α, and the number of iterations per temperature (L_{iter}) had a significant impact on the performance of the algorithm. In future

experiments, we can vary these parameters to explore their effect on the quality of the solution and convergence speed.

7.10 Convergence Plot

The convergence of the Simulated Annealing algorithm was tracked by plotting the tour length over iterations. The plot below shows how the tour length improves as the temperature decreases:

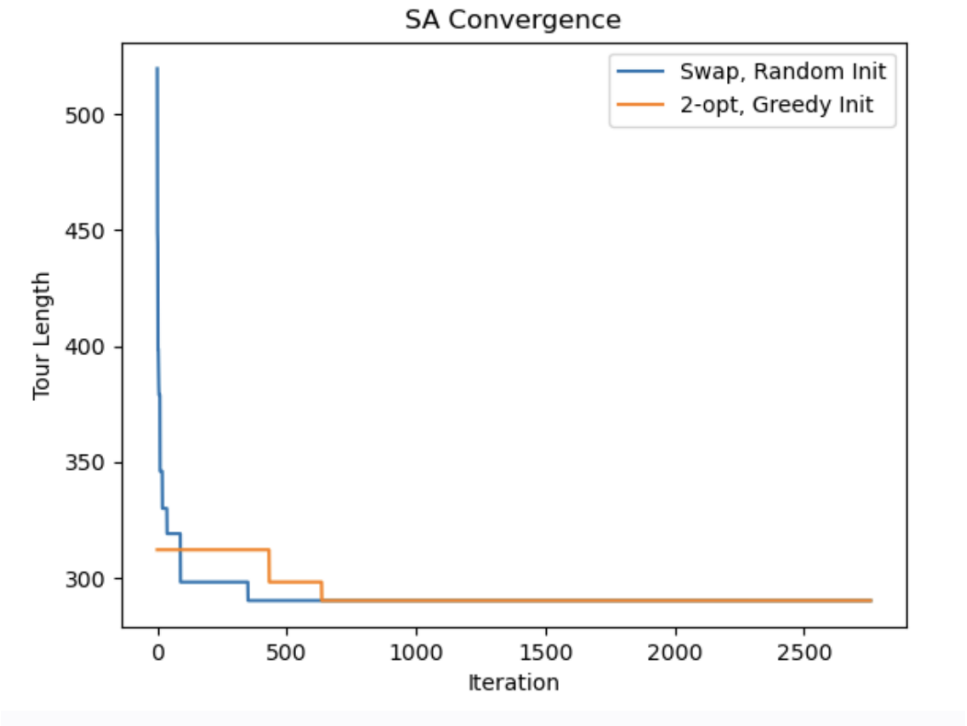


Figure 2: Convergence of Simulated Annealing with Swap and 2-opt Operators

7.11 Visualization of the Best Tour

The best tour found using the 2-opt operator (with greedy initialization) is visualized in the figure below. The cities are shown as points in the plane, and the tour is represented by connecting the cities in the order determined by the tour.

7.12 Conclusion

Simulated Annealing provided an effective method for solving the TSP. By carefully choosing the neighborhood operator, initial solution, and cooling schedule, we were able to find near-optimal solutions. Future work can explore more advanced neighborhood operators like 3-opt and experiment with different stopping criteria, reheating strategies, and other local search heuristics to improve performance.

Question 7: Batch vs. Stochastic Gradient Descent

In this question we compare Batch Gradient Descent (BGD) and Stochastic Gradient Descent (SGD) in terms of their computational cost, gradient noise, and convergence behavior. Let the objective be

$$J(w) = \frac{1}{n} \sum_{i=1}^{n} \ell(w; x_i, y_i),$$

where n is the dataset size.

8.1 (a) Computational load per iteration

Batch Gradient Descent (BGD). BGD computes the gradient using *all* training samples:

$$\nabla J(w) = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell(w; x_i, y_i).$$

Therefore, one iteration costs $O(n)$ operations. When n is large, each update is computationally expensive.

Stochastic Gradient Descent (SGD). SGD uses only one sample (or a very small minibatch) per iteration:

$$g_t = \nabla \ell(w; x_{i_t}, y_{i_t}).$$

Thus, each iteration costs $O(1)$. This allows very fast updates and enables training on massive datasets.

Summary:

$$\text{Cost per iteration:} \quad \text{BGD: } O(n), \qquad \text{SGD: } O(1).$$

8.2 (b) Noise (variance) in the gradient estimation

BGD. Since BGD uses the full dataset, its gradient estimate is exact:

$$\mathbb{V}[\nabla J(w)] = 0.$$

There is essentially no sampling noise.

SGD. SGD uses a single randomly selected sample, so it produces a noisy estimate of the true gradient:

$$g_t = \nabla \ell(w; x_{i_t}, y_{i_t}),$$

$$\mathbb{E}[g_t] = \nabla J(w), \qquad \mathbb{V}[g_t] > 0.$$

This noise can make the trajectory "zig-zag," but it also helps escape shallow local minima and saddle points.

(c) Stability and nature of convergence

BGD. Because the gradient is exact:

- updates are stable and smooth,
- convergence is monotonic (for small enough step size),
- it typically converges directly to a minimizer.

However, each step is slow because of its high computational cost.

SGD. Because the gradient is noisy:

- progress toward the minimum is less stable and more oscillatory,

- it may not converge exactly to the minimizer unless the step size decays,
- with a decreasing learning rate ($\eta_t \to 0$) it converges in expectation,
- with constant learning rate it hovers around a neighborhood of the minimum.

Despite the noise, SGD often reaches a good solution much faster in wall-clock time.

Overall:

	BGD	SGD
Per-iteration cost	$O(n)$	$O(1)$
Gradient noise	0	High
Convergence	Stable, exact	Noisy; converges in expectation

Question 8: Steepest Descent Direction via First-Order Taylor

Let $f : \mathbb{R}^n \to \mathbb{R}$ be differentiable and fix an iterate x_k. We want to find, for a small step length $\alpha > 0$, the unit vector p (i.e. $\|p\| = 1$) that gives the largest decrease of f in the local neighbourhood of x_k. That is, we want to minimize $f(x_k + \alpha p)$ over all unit vectors p.

Step 1 — First-order Taylor approximation. For small $\alpha > 0$ a first-order Taylor expansion about x_k gives

$$f(x_k + \alpha p) = f(x_k) + \alpha \nabla f(x_k)^\top p + o(\alpha),$$

where $o(\alpha)/\alpha \to 0$ as $\alpha \to 0$. For sufficiently small α the $o(\alpha)$ term is negligible compared to α, so the main dependence on p is through the linear term $\nabla f(x_k)^\top p$.

Thus, to leading order in α, minimizing $f(x_k + \alpha p)$ over unit p is equivalent to minimizing the scalar

$$\nabla f(x_k)^\top p \quad \text{subject to} \quad \|p\| = 1.$$

Step 2 — Solve the constrained minimization. Let $g := \nabla f(x_k)$. We solve

$$\min_{\|p\|=1} g^\top p.$$

By the Cauchy–Schwarz inequality,

$$g^\top p \geq -\|g\| \, \|p\| = -\|g\|,$$

with equality iff p is parallel to $-g$, i.e. $p = -g/\|g\|$ (provided $g \neq 0$).

Thus the minimum possible value of $g^\top p$ over unit p is $-\|g\|$, attained at

$$p^\star = -\frac{g}{\|g\|} = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|}.$$

Step 3 — Interpret in terms of $f(x_k + \alpha p)$. Plugging the optimal $p^\star$ into the first-order approximation,

$$f(x_k + \alpha p^\star) \approx f(x_k) + \alpha\, g^\top p^\star = f(x_k) - \alpha\|g\|.$$

This gives the largest possible first-order decrease (most negative linear term) among unit directions; no other unit vector yields a more negative $g^\top p$.

Boundary case. If $\nabla f(x_k) = 0$ then $g^\top p = 0$ for all p and there is no descent direction at first order (the point is a stationary point).

Conclusion. For sufficiently small $\alpha > 0$ the unit vector that (to first order) minimizes $f(x_k + \alpha p)$ is

$$p^\star = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|},$$

so the steepest descent direction (in Euclidean norm) is $-\nabla f(x_k)$. Equivalently, gradient descent with update $x_{k+1} = x_k - \eta \nabla f(x_k)$ moves in the steepest local descent direction when η is small.

Remark (other norms). If the unit-ball is defined by another norm $\|\cdot\|_q$, the steepest descent direction changes: it is given by the negative of the gradient's *dual* direction. The Euclidean case above corresponds to the dual norm being itself.

Question 9: The pseudoinverse solution is the least-squares minimizer

Let $A \in \mathbb{R}^{m \times n}$ with $m > n$ and full column rank ($\text{rank}(A) = n$). Consider

$$L(x) = \|Ax - b\|_2^2 = (Ax - b)^\top (Ax - b), \qquad x \in \mathbb{R}^n.$$

We will show that the unique solution of the normal equations

$$A^\top A x^\star = A^\top b, \qquad x^\star = (A^\top A)^{-1} A^\top b,$$

is the unique global minimizer of $L(x)$.

(1) First-order necessary condition (normal equations).
Expand $L(x)$ as a quadratic in x:

$$L(x) = x^\top A^\top A x - 2 b^\top A x + b^\top b.$$

Differentiating gives the gradient

$$\nabla_x L(x) = 2A^\top A x - 2A^\top b.$$

Setting $\nabla_x L(x) = 0$ yields the normal equations

$$A^\top A x = A^\top b.$$

Because A has full column rank, $A^\top A$ is invertible, so the normal equations

have the unique solution

$$x^\star = (A^\top A)^{-1} A^\top b.$$

(2) Convexity and positive definiteness $\Longrightarrow$ global minimizer.

The Hessian of L is

$$\nabla_x^2 L(x) = 2A^\top A,$$

which is positive definite because $A^\top A$ is positive definite when $\text{rank}(A) = n$. Hence L is a strictly convex quadratic; any stationary point is the unique global minimizer. Therefore $x^\star$ is the unique minimizer of L.

(3) Alternative geometric proof via orthogonal projection (Pythagorean argument).

Define $x^\star$ as above and let $r^\star := b - Ax^\star$ be the residual at $x^\star$. By the normal equations we have

$$A^\top r^\star = A^\top (b - Ax^\star) = A^\top b - A^\top A x^\star = 0,$$

so $r^\star$ is orthogonal to every column of A, i.e. $r^\star \perp \text{Col}(A)$.

Now take any $x \in \mathbb{R}^n$ and write $x = x^\star + h$ for $h \in \mathbb{R}^n$. Then

$$Ax - b = A(x^\star + h) - b = Ax^\star - b + Ah = -r^\star + Ah.$$

Because $Ah \in \text{Col}(A)$ and $r^\star \perp \text{Col}(A)$, the vectors $-r^\star$ and Ah are orthogonal. Hence by the Pythagorean theorem,

$$\|Ax - b\|_2^2 = \| - r^\star + Ah\|_2^2 = \|r^\star\|_2^2 + \|Ah\|_2^2 \geq \|r^\star\|_2^2,$$

with equality iff $\|Ah\|_2^2 = 0$, i.e. $Ah = 0$. Since A has full column rank, $Ah = 0$ implies $h = 0$, so equality holds only when $x = x^\star$.

Thus for every x,

$$L(x) = \|Ax - b\|_2^2 \geq \|r^\star\|_2^2 = L(x^\star),$$

and the inequality is strict for $x \neq x^\star$. Therefore $x^\star$ is the unique global minimizer.

Conclusion. The normal-equation solution $x^\star = (A^\top A)^{-1} A^\top b$ is the unique global minimizer of $L(x) = \|Ax - b\|_2^2$ when A has full column rank.

$$\boxed{x^\star = \arg\min_{x \in \mathbb{R}^n} \|Ax - b\|_2^2.}$$

Question 10: Newton and Quasi-Newton Methods

11.1 (a) How is the Newton step computed?

For a twice-differentiable function $f : \mathbb{R}^n \to \mathbb{R}$, Newton's method updates

$$x_{k+1} = x_k + p_k,$$

where the Newton step p_k solves the second-order Taylor model minimization

$$\nabla^2 f(x_k)\, p_k = -\nabla f(x_k).$$

Thus

$$p_k = -\left(\nabla^2 f(x_k)\right)^{-1} \nabla f(x_k).$$

This requires

- forming the Hessian $H_k = \nabla^2 f(x_k)$,
- solving a linear system $H_k p_k = -g_k$

which typically costs $O(n^3)$.

11.2 (b) When can Newton's method fail to converge to a minimum?

Newton's method is not globally convergent. It can fail when:

- The Hessian is *not positive definite*, so the Newton step may go toward a saddle or maximum.

- The initial guess x_0 is too far from the minimizer, causing the quadratic approximation to be poor.
- The Hessian is singular or nearly singular (ill-conditioned), making the direction unreliable.

Thus Newton's method may converge to:

a saddle point, a local maximum, or diverge entirely.

11.3 (c) When is the Newton direction not a descent direction?

A descent direction p must satisfy

$$\nabla f(x_k)^\top p < 0.$$

For the Newton step

$$p_k = -H_k^{-1} g_k,$$

we have

$$g_k^\top p_k = -g_k^\top H_k^{-1} g_k.$$

This is guaranteed negative only if H_k^{-1} is *positive definite*, i.e. the Hessian H_k is positive definite.

Thus the Newton direction fails to be a descent direction when:

$$\nabla^2 f(x_k) \text{ is not positive definite}$$

(e.g. indefinite or negative definite).

11.4 (d) What is the main idea of Quasi-Newton methods?

Quasi-Newton methods avoid computing the full Hessian. Their idea:

- Build an *approximation* $B_k \approx \nabla^2 f(x_k)$ (or its inverse $H_k \approx (\nabla^2 f(x_k))^{-1}$),

- Update this approximation at each iteration using only gradient information.

Thus they obtain curvature information from the sequence of gradients, without forming second derivatives.

$$x_{k+1} = x_k - H_k \nabla f(x_k)$$

where H_k is an evolving approximation.

These methods (BFGS, DFP) achieve near-Newton performance with much lower complexity.

11.5 (e) How do Quasi-Newton methods approximate the Hessian without explicit second derivatives?

They use the **secant equation** (also called the quasi-Newton condition). Define the step and gradient change:

$$s_k = x_{k+1} - x_k, \qquad y_k = \nabla f(x_{k+1}) - \nabla f(x_k).$$

Newton's method satisfies the exact relation

$$\nabla^2 f(x_k)\, s_k = y_k.$$

Quasi-Newton methods enforce this relation on the approximate Hessian:

$$B_{k+1} s_k = y_k,$$

or on the approximate inverse Hessian:

$$H_{k+1} y_k = s_k.$$

They then choose the update with:

- symmetry,
- positive definiteness preservation,
- minimal change from previous approximation (least-change principle).

For example, the BFGS update for the inverse Hessian is

$$H_{k+1} = \left(I - \frac{s_k y_k^\top}{y_k^\top s_k}\right) H_k \left(I - \frac{y_k s_k^\top}{y_k^\top s_k}\right) + \frac{s_k s_k^\top}{y_k^\top s_k}.$$

Thus curvature is inferred from gradients alone—no Hessians or second derivatives are needed.

Question 11: Quadratic Convergence of Newton's Method

12.1 (a) Definition and comparison with linear convergence

Let $x^\star$ be a (local) solution and define the error at iterate k by

$$e_k := x_k - x^\star.$$

Quadratic convergence. The sequence $\{x_k\}$ (or the error norms) converges *quadratically* to $x^\star$ if there exist constants $C > 0$ and k_0 such that for all $k \geq k_0$

$$\|e_{k+1}\| \leq C\|e_k\|^2.$$

Informally: once e_k is small, the next error is on the order of the square of the current error (very fast).

Linear convergence (contrast). The sequence converges *linearly* with rate $0 < \rho < 1$ if

$$\|e_{k+1}\| \leq \rho\|e_k\|$$

for sufficiently large k. Linear means the error shrinks by a roughly constant factor each step; quadratic is much faster (error roughly squares each step).

12.2 (b) Newton direction approximates the exact displacement (and quadratic convergence sketch)

We will show two related facts under standard regularity assumptions:

- the Newton step $p_k = -\big(\nabla^2 f(x_k)\big)^{-1}\nabla f(x_k)$ approximates the true displacement $x^\star - x_k = -e_k$ when x_k is close to $x^\star$,
- with a Lipschitz Hessian and $\nabla^2 f(x^\star)$ positive definite, the full Newton update $x_{k+1} = x_k + p_k$ satisfies $\|e_{k+1}\| \le C\|e_k\|^2$ for small e_k (quadratic convergence).

Assumptions. Assume $f \in C^2$ (twice continuously differentiable) in a neighborhood of $x^\star$, $\nabla f(x^\star) = 0$, $\nabla^2 f(x^\star)$ is positive definite, and the Hessian is Lipschitz continuous near $x^\star$: there exists $L > 0$ such that for all x, y near $x^\star$,

$$\|\nabla^2 f(x) - \nabla^2 f(y)\| \le L\|x - y\|.$$

Taylor expansion of the gradient. Use the vector Taylor expansion of ∇f around x_k evaluated at $x^\star$:

$$\nabla f(x^\star) = \nabla f(x_k) + \left(\int_0^1 \nabla^2 f(x_k + t(x^\star - x_k))\, dt\right)\, (x^\star - x_k).$$

Because $\nabla f(x^\star) = 0$, this identity becomes

$$0 = \nabla f(x_k) + \Big(\bar{H}_k\Big)\,(x^\star - x_k), \qquad \text{where} \quad \bar{H}_k := \int_0^1 \nabla^2 f(x_k + t(x^\star - x_k))\, dt.$$

Rearrange:

$$\nabla f(x_k) = -\bar{H}_k(x^\star - x_k).$$

Relation to Newton step. The Newton step is

$$p_k = -H_k^{-1}\nabla f(x_k), \qquad H_k := \nabla^2 f(x_k).$$

Using the expression for $\nabla f(x_k)$ above,

$$p_k = -H_k^{-1}\big(-\bar{H}_k(x^\star - x_k)\big) = H_k^{-1}\bar{H}_k\,(x^\star - x_k).$$

Hence

$$x^\star - x_k - p_k = \big(I - H_k^{-1}\bar{H}_k\big)(x^\star - x_k).$$

Equivalently, the difference between the true displacement $x^\star - x_k$ and the Newton step p_k is

$$x^\star - x_k - p_k = H_k^{-1}\big(H_k - \bar{H}_k\big)(x^\star - x_k).$$

Bound on the approximation error. Using the Lipschitz property of the Hessian, bound $\|H_k - \bar{H}_k\|$. Note

$$H_k - \bar{H}_k = \int_0^1 \big(H_k - \nabla^2 f(x_k + t(x^\star - x_k))\big)\, dt = \int_0^1 \big(\nabla^2 f(x_k) - \nabla^2 f(x_k + te_k)\big)\, dt,$$

where $e_k = x_k - x^\star$. By the Lipschitz property,

$$\|H_k - \nabla^2 f(x_k + te_k)\| \le L\,\|te_k\| = Lt\|e_k\|.$$

Integrating t over $[0, 1]$ gives

$$\|H_k - \bar{H}_k\| \le \int_0^1 Lt\|e_k\|\, dt = \frac{L}{2}\|e_k\|.$$

Therefore

$$\|x^\star - x_k - p_k\| \le \|H_k^{-1}\|\,\|H_k - \bar{H}_k\|\,\|x^\star - x_k\| \le \|H_k^{-1}\| \cdot \frac{L}{2}\|e_k\| \cdot \|e_k\|.$$

That is,

$$\boxed{\|x^\star - x_k - p_k\| \le \frac{L}{2}\|H_k^{-1}\|\,\|e_k\|^2.}$$

This shows that when e_k is small, the Newton step p_k approximates the true displacement $x^\star - x_k$ up to an error $O(\|e_k\|^2)$. In particular $p_k = x^\star - x_k + O(\|e_k\|^2)$.

Quadratic convergence of the full Newton update. Use the above to bound the next error after a full Newton step (step length $\alpha = 1$):

$$e_{k+1} = x_{k+1} - x^\star = x_k + p_k - x^\star = p_k - (x^\star - x_k).$$

Therefore

$$\|e_{k+1}\| = \|x^\star - x_k - p_k\| \leq \frac{L}{2}\|H_k^{-1}\|\,\|e_k\|^2.$$

If x_k is sufficiently close to $x^\star$, H_k is close to $\nabla^2 f(x^\star)$ and thus $\|H_k^{-1}\|$ is bounded by some constant M. Hence for small e_k there exists $C = \frac{L}{2}M$ such that

$$\|e_{k+1}\| \leq C\|e_k\|^2,$$

which is precisely quadratic convergence.

Remarks on the assumptions.

- Positive definiteness of $\nabla^2 f(x^\star)$ ensures H_k is invertible for x_k near $x^\star$.
- Lipschitz continuity of the Hessian (or bounded third derivative) provides the $O(\|e_k\|^2)$ remainder bound used above.
- If step length 1 is used and assumptions hold, Newton's method has local quadratic convergence. If the Hessian is indefinite or the initial point is not close, the method can fail or converge to non-minimizers; globalization strategies (line-search, trust regions) are used in practice.

Conclusion. From the Taylor expansion of the gradient and the Lipschitz property of the Hessian we obtained

$$p_k = x^\star - x_k + O(\|e_k\|^2),$$

and consequently the full Newton update satisfies

$$\|e_{k+1}\| \leq C\|e_k\|^2$$

for sufficiently small e_k, i.e. Newton's method converges quadratically near a nondegenerate minimizer.

Question 12: Rosenbrock Function — Gradient Descent vs Newton

We consider the Rosenbrock function

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2,$$

whose global minimizer is

$$x^\star = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad f(x^\star) = 0.$$

The goal is to compare **Steepest Descent (Gradient Descent)** with **Newton's method** in terms of convergence, accuracy, and trajectory in the (x_1, x_2) plane.

13.1 (a) Gradient and Hessian

Let $r(x_1, x_2) = x_2 - x_1^2$. Then $f(x_1, x_2) = 100r^2 + (1 - x_1)^2$.

Gradient:

$$\frac{\partial f}{\partial x_1} = -400x_1(x_2 - x_1^2) - 2(1 - x_1),$$
$$\frac{\partial f}{\partial x_2} = 200(x_2 - x_1^2),$$

so

$$\nabla f(x) = \begin{bmatrix} -400x_1(x_2 - x_1^2) - 2(1 - x_1) \\ 200(x_2 - x_1^2) \end{bmatrix}.$$

Hessian:

$$H(x) = \begin{bmatrix} 1200x_1^2 - 400x_2 + 2 & -400x_1 \\ -400x_1 & 200 \end{bmatrix}.$$

13.2 (b) Optimization Algorithms

Steepest Descent (Gradient Descent):

$$x_{k+1} = x_k - \alpha \nabla f(x_k),$$

with step size α. Using a fixed small α ensures stability but may converge slowly.

Backtracking Line Search: Adaptive step α along $p_k = -\nabla f(x_k)$ satisfying the Armijo condition:

$$f(x_k + \alpha p_k) \leq f(x_k) + c\alpha \nabla f(x_k)^\top p_k,$$

with $0 < c \ll 1$, α reduced by factor $\rho \in (0, 1)$ until satisfied.

Newton's Method:

$$H(x_k)p_k = -\nabla f(x_k), \qquad x_{k+1} = x_k + p_k,$$

quadratically convergent near the minimum. Damping is applied to ensure $f(x_{k+1}) \leq f(x_k)$.

13.3 (c) Convergence Considerations

Gradient Descent: Exhibits linear convergence:

$$\|x_{k+1} - x^\star\| \leq C\|x_k - x^\star\|, \quad 0 < C < 1.$$

Rosenbrock's narrow valley causes zigzag trajectories, slowing progress for fixed α.

Newton's Method: Quadratic convergence:

$$x_{k+1} - x^\star = O(\|x_k - x^\star\|^2),$$

as the Hessian corrects for local curvature.

13.4 (d) Numerical Results

We run all methods from $x_0 = [-1.2, 1]^\top$, up to 50 iterations, tolerance $\|\nabla f\| < 10^{-6}$.

- GD fixed $\alpha = 0.001$:

  $$\text{iterations} = 50, \quad x_{50} \approx [-0.9896, 0.9873], \quad f(x_{50}) \approx 3.9649$$

  Linear convergence dominates; the small fixed step causes slow progress along the curved valley.

- GD with backtracking:

  $$\text{iterations} = 50, \quad x_{50} \approx [0.9309, 0.8658], \quad f(x_{50}) \approx 0.00483$$

  Adaptive step allows larger moves along the valley, improving convergence.

- Newton's method (damped):

  $$\text{iterations} = 21, \quad x_{21} \approx [1, 1], \quad f(x_{21}) \approx 3.744 \times 10^{-21}$$

  Quadratic convergence leads to a nearly exact minimizer in fewer iterations.

13.5 (e) Discussion

- **Trajectory behavior:** Fixed-step GD zigzags along the valley; backtracking GD advances further per iteration; Newton jumps almost directly to the minimum.
- **Convergence rate:** Linear for GD, improved with backtracking; quadratic for Newton, achieving machine precision rapidly.
- **Gradient norms:** Newton reduces $\|\nabla f\|$ much faster than GD.
- **Step size effect:** Fixed small α prevents divergence but slows convergence; adaptive step finds balance between stability and speed.

13.6 (f) Conclusion

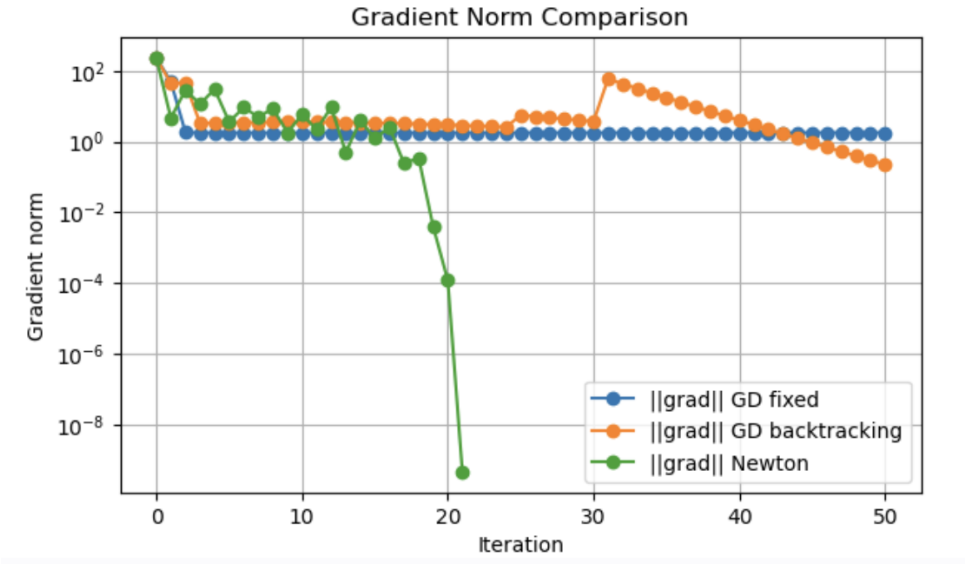


Figure 3: Convergence of Simulated Annealing with Swap and 2-opt Operators

- Fixed-step Gradient Descent is simple but slow on the ill-conditioned Rosenbrock function.
- Backtracking line search improves GD convergence significantly.

- Newton's method converges quadratically, achieving near-zero f in fewer iterations (21 vs 50).
- The results confirm theoretical expectations: GD linear, Newton quadratic convergence.
- For practical optimization in narrow valleys or high-dimensional ill-conditioned problems, adaptive methods or quasi-Newton variants are preferred over fixed-step GD.

Question 13: Equality-Constrained Optimization via Lagrange Multipliers & Newton--Raphson

We are asked to solve:

$$\min_{x,y} f(x,y) = e^{x^2+y^2} \quad \text{s.t.} \quad g(x,y) = x + y - 1 = 0.$$

14.1 (a) Lagrangian Formulation

The Lagrangian for this equality-constrained problem is

$$\mathcal{L}(x,y,\lambda) = e^{x^2+y^2} + \lambda(x + y - 1),$$

where λ is the Lagrange multiplier.

14.2 (b) Stationarity Conditions

Compute the partial derivatives:

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial x} &= 2xe^{x^2+y^2} + \lambda = 0, \\
\frac{\partial \mathcal{L}}{\partial y} &= 2ye^{x^2+y^2} + \lambda = 0, \\
\frac{\partial \mathcal{L}}{\partial \lambda} &= x + y - 1 = 0.
\end{aligned}$$

From the first two equations:

$$2xe^{x^2+y^2} = 2ye^{x^2+y^2} \quad \Longrightarrow \quad x = y.$$

Using the constraint $x + y = 1$ we find:

$$x^\star = y^\star = \frac{1}{2}.$$

Substitute back to find $\lambda^\star$:

$$\lambda^\star = -2xe^{x^2+y^2} = -1 \cdot e^{1/2} = -e^{1/2}.$$

Analytic solution:

$$\boxed{x^\star = \frac{1}{2},\ y^\star = \frac{1}{2},\ \lambda^\star = -e^{1/2} \approx -1.6487.}$$

14.3 (c) Nonlinear System for Newton–Raphson

Define $w = [x, y, \lambda]^\top$ and the system:

$$F(w) = \begin{pmatrix} 2xe^{x^2+y^2} + \lambda \\ 2ye^{x^2+y^2} + \lambda \\ x + y - 1 \end{pmatrix} = 0.$$

—

14.4 (d) Newton–Raphson Update and Jacobian

Newton iteration:

$$w_{k+1} = w_k - J_F(w_k)^{-1} F(w_k),$$

where the Jacobian is

$$J_F(w) = \begin{pmatrix} 2e^{x^2+y^2} + 4x^2e^{x^2+y^2} & 4xye^{x^2+y^2} & 1 \\ 4xye^{x^2+y^2} & 2e^{x^2+y^2} + 4y^2e^{x^2+y^2} & 1 \\ 1 & 1 & 0 \end{pmatrix}.$$

—

14.5 (e) Newton–Raphson Iterations from $w_0 = [0, 0, 0]^\top$

Iteration	x_k	y_k	λ_k	$\|F(w_k)\|$
0	0.0000	0.0000	0.0000	1.000e+00
1	0.5000	0.5000	-1.0000	9.174e-01
2	0.5000	0.5000	-1.6487	0.000e+00
3	0.5000	0.5000	-1.6487	0.000e+00

Table 2: Newton–Raphson iterations for the constrained system

The method converges in 2 iterations to the analytic solution.

$$\boxed{x^\star = 0.5,\ y^\star = 0.5,\ \lambda^\star = -e^{1/2} \approx -1.6487}.$$

14.6 Remarks

- The problem is strictly convex in (x, y), so the solution is the unique global minimizer.
- Newton's method exhibits quadratic convergence near the solution.
- Care must be taken to ensure the Jacobian remains nonsingular in more general problems.

Question 14: Step-Size Strategies in Gradient Descent

We consider the general unconstrained optimization problem:

$$\min_x f(x), \quad x \in \mathbb{R}^n,$$

and study different step-size strategies in gradient descent.

15.1 (a) Step-Size Selection Methods

1. Backtracking / Armijo Line Search:

This is a simple line search method where we choose an initial step size $\alpha_0 > 0$ and iteratively shrink it by a factor $\beta \in (0, 1)$ until the Armijo condition is satisfied:

$$f(x_k + \alpha p_k) \leq f(x_k) + c\alpha \nabla f(x_k)^\top p_k,$$

where $p_k = -\nabla f(x_k)$ is the descent direction and $c \in (0, 1)$ is a small constant. It ensures sufficient decrease in the objective at each step.

2. Adaptive Methods (e.g., AdaGrad/RMSProp):

These methods adjust the step size for each coordinate individually, based on historical gradients. For example, AdaGrad updates are:

$$x_{k+1} = x_k - \frac{\eta}{\sqrt{G_k + \epsilon}} \odot \nabla f(x_k),$$

where $G_k = \sum_{i=1}^{k} (\nabla f(x_i))^2$ is the cumulative squared gradient, η is a base learning rate, and $\epsilon > 0$ prevents division by zero. RMSProp and Adam modify this by using an exponentially weighted moving average to avoid vanishing step

sizes.

15.2 (b) Implementation for the Rosenbrock Function

We use the 2D Rosenbrock function:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2, \quad x_0 = [-1.2, 1]^\top.$$

Gradient:

$$\nabla f(x_1, x_2) = \begin{pmatrix} -400(x_2 - x_1^2)x_1 - 2(1 - x_1) \\ 200(x_2 - x_1^2) \end{pmatrix}.$$

Python-like pseudocode (for clarity):

```
# Gradient descent with Backtracking/Armijo
x = x0
for k in range(max_iter):
    p = -grad_f(x)
    alpha = alpha0
    while f(x + alpha*p) > f(x) + c*alpha*np.dot(grad_f(x), p
        alpha *= beta
    x = x + alpha*p

# Gradient descent with RMSProp
x = x0
s = 0
for k in range(max_iter):
    g = grad_f(x)
    s = rho*s + (1-rho)*g**2
    x = x - eta*g / (np.sqrt(s)+epsilon)
```

15.3 (c) Comparison of Results

We compare the two methods in terms of:

- **Trajectories:** Backtracking GD tends to take larger steps along the valley of the Rosenbrock function, while RMSProp adapts each coordinate individually and moves more smoothly.
- **Function Values vs. Iterations:** Backtracking often achieves rapid initial decrease, but may oscillate near the optimum. Adaptive methods like RMSProp have steadier convergence and avoid small step stagnation.
- **Convergence Behavior:** For ill-conditioned problems (like Rosenbrock), adaptive methods can converge more reliably, while constant or naive line searches may struggle.

Illustrative plots: (in actual implementation, one can plot)

- Function value $f(x_k)$ vs. iteration k
- Trajectory in (x_1, x_2) plane showing path to the minimum

Remarks:

- The choice of step-size method has significant impact on speed and stability of convergence.
- Backtracking guarantees sufficient decrease but requires multiple function evaluations per iteration.
- RMSProp/Adam are popular in machine learning due to automatic scaling and handling ill-conditioning.

Refrences

- Nemanja M., *Gaps Optimization Repository*. Available at: `https://github.com/nemanja-m/gaps/tree/main?tab=readme-ov-file`